

2.2 첫 번째 머신러닝 만들어 보기 — 붓꽃 품종 예측하기

목차

1. 붓꽃 품종 예측 문제 정의
 1. 지도학습과 분류(Classification)
 2. 붓꽃 데이터 세트 소개
 3. 머신러닝 프로세스 개요
2. 필요한 모듈 임포트
3. 데이터 세트 로딩과 확인
4. 학습 데이터와 테스트 데이터 분리 - `train_test_split()`
5. 모델 학습 - `fit()`
6. 예측 수행 - `predict()`
7. 성능 평가 - `accuracy_score()`
8. 핵심 요약 및 머신러닝 활용 포인트

1. 붓꽃 품종 예측 문제 정의

1.1 지도학습과 분류(Classification)

이번 장에서는 사이킷런을 이용해 붓꽃(Iris) 데이터 세트로 붓꽃의 품종을 분류(Classification)하는, 가장 대표적인 머신러닝 입문 예제를 다룹니다. 분류는 지도학습(Supervised Learning)의 대표적인 유형입니다.

지도학습은 학습을 위한 다양한 피처와 분류 결정값인 레이블(Label) 데이터로 모델을 학습한 뒤, 별도의 테스트 데이터 세트에서 미지의 레이블을 예측하는 방식입니다. 즉 명확한 정답이 주어진 데이터를 먼저 학습한 뒤 미지의 정답을 예측하는 방식입니다.

용어	설명
피처(Feature)	데이터 세트의 속성. 붓꽃 데이터에서는 꽃잎의 길이·너비, 꽃받침의 길이·너비 4가지입니다. 관례상 X 로 표기합니다.
레이블(Label)	예측해야 할 정답 데이터. 결정값, 타깃값, 클래스로도 부릅니다. 관례상 y 로 표기합니다.
학습 데이터 세트	모델을 학습시키는 데 사용하는 데이터(피처 + 레이블).
테스트 데이터 세트	학습된 모델의 성능을 평가하는 데 사용하는, 학습에 쓰이지 않은 별도 데이터.

1.2 붓꽃 데이터 세트 소개

붓꽃 데이터 세트는 꽃받침(sepal)의 길이·너비, 꽃잎(petal)의 길이·너비 4개 피처를 기반으로 붓꽃의 품종을 예측하는 데이터입니다. 총 150개의 샘플로 구성되며, 품종은 Setosa, Versicolor, Virginica 3가지가 각각 50개씩 균등하게 들어 있습니다.

1.3 머신러닝 프로세스 개요

사이킷런을 이용한 분류 예측 프로세스는 아래 5단계로 요약됩니다. 이 흐름은 어떤 알고리즘을 쓰더라도 동일하므로, 이 장에서 반드시 체득해야 할 핵심입니다.

단계	내용	사용 API
① 데이터 세트 분리	데이터를 학습용과 테스트용으로 분리	<code>train_test_split()</code>
② 모델 학습	학습 데이터를 기반으로 알고리즘이 규칙을 학습	<code>fit()</code>
③ 예측 수행	학습된 모델로 테스트 데이터의 레이블 예측	<code>predict()</code>
④ 평가	예측 결과와 실제 정답을 비교해 성능 측정	<code>accuracy_score()</code>

2. 필요한 모듈 импорт

```
from sklearn.datasets import load_iris
from sklearn.tree import DecisionTreeClassifier
from sklearn.model_selection import train_test_split
```

줄	코드	상세 설명
1	<code>from sklearn.datasets import load_iris</code>	<code>sklearn.datasets</code> 모듈은 사이킷런에 내장된 예제 데이터 세트를 제공합니다. <code>load_iris</code> 는 그중 붓꽃 데이터를 로딩하는 함수입니다. 별도의 파일 다운로드 없이 즉시 사용할 수 있습니다.

2	from sklearn.tree import DecisionTreeClassifier	<code>sklearn.tree</code> 모듈은 트리 기반 ML 알고리즘 을 제공합니다. <code>DecisionTreeClassifier</code> 는 결정 트리 분류기 로, 데이터의 규칙을 if-else 형태 의 트리 구조로 학습하는 직관적인 알고리즘입니다.
3	from sklearn.model_selection import train_test_split	<code>sklearn.model_selection</code> 은 학습/테스트 데이터 분리, 교차 검증, 하이퍼 파라미터 튜닝 등을 지원하는 모듈입니다. <code>train_test_split</code> 은 데이터를 학습용과 테스트용으로 나누는 함수입니다.

■ 사이킷런 모듈 명명 규칙

사이킷런은 `sklearn.기능영역` 형태로 모듈이 구성되어 있습니다. `datasets` (예제 데이터), `preprocessing` (전처리), `model_selection` (모델 선택/평가), `metrics` (평가 지표), `tree / ensemble / linear_model` (알고리즘) 등 **이름만 봐도 역할을 알 수 있도록** 설계되어 있습니다.

3. 데이터 세트 로딩과 확인

```
import pandas as pd

# 붓꽃 데이터 세트를 로딩합니다.
iris = load_iris()

# iris.data는 Iris 데이터 세트에서 피처(feature)만으로 된 데이터를 numpy로 가지고 있습니다.
iris_data = iris.data

# iris.target은 붓꽃 데이터 세트에서 레이블(결정 값) 데이터를 numpy로 가지고 있습니다.
iris_label = iris.target
print('iris target값:', iris_label)
print('iris target명:', iris.target_names)

# 붓꽃 데이터 세트를 자세히 보기 위해 DataFrame으로 변환합니다.
iris_df = pd.DataFrame(data=iris_data, columns=iris.feature_names)
iris_df['label'] = iris.target
iris_df.head(3)
```

줄	코드	상세 설명
1	import pandas as pd	로딩한 데이터를 표 형태로 보기 위해 판다스를 임포트합니다.
4	iris = load_iris()	붓꽃 데이터 세트를 로딩합니다. 반환 객체는 파이썬 딕셔너리와 유사한 Bunch 객체 로, 피처·레이블·이름·설명 등을 한꺼번에 담고 있습니다.
7	iris_data = iris.data	<code>data</code> 속성은 피처 데이터만 담은 numpy ndarray입니다. shape는 (150, 4) 로 150개 샘플, 4개 피처입니다.
10	iris_label = iris.target	<code>target</code> 속성은 레이블(정답) 데이터 입니다. shape는 (150,) 인 1차원 배열이며, 값은 0/1/2로 이미 숫자로 인코딩 되어 있습니다.
11	print('iris target값:', iris_label)	0이 50개, 1이 50개, 2가 50개 순서대로 정렬되어 있음을 확인할 수 있습니다. 데이터가 레이블 순으로 정렬 되어 있다는 이 사실은 뒤의 교차 검증(2.4장)에서 매우 중요해집니다.
12	print('iris target명:', iris.target_names)	<code>target_names</code> 는 숫자 레이블에 대응하는 실제 품종 이름 입니다. 0=setosa, 1=versicolor, 2=virginica로 매핑됩니다.

15	<code>iris_df = pd.DataFrame(data=iris_data, columns=iris.feature_names)</code>	ndarray인 피쳐 데이터를 DataFrame으로 변환합니다. <code>columns</code> 에 <code>feature_names</code> 를 넣어 의미 있는 컬럼명 을 부여합니다.
16	<code>iris_df['label'] = iris.target</code>	레이블을 <code>label</code> 이라는 새 컬럼으로 추가하여 피쳐와 정답을 한 표에서 볼 수 있게 합니다.
17	<code>iris_df.head(3)</code>	상위 3건을 확인합니다. 모두 label=0(setosa)이며 꽃잎 길이가 1.4 내외로 짧은 것이 특징입니다.

▶ 실행 결과

```
iris target값: [0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0  
0 0 0 0 0 0 0 0 0 0 0 0 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1  
1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 2 2 2 2 2 2 2 2 2  
2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2  
2 2]  
iris target명: ['setosa' 'versicolor' 'virginica']
```

	sepal length (cm)	sepal width (cm)	petal length (cm)	petal width (cm)	label
0	5.1	3.5	1.4	0.2	0
1	4.9	3.0	1.4	0.2	0
2	4.7	3.2	1.3	0.2	0

■ 결과 해석

피쳐는 모두 **연속형 숫자(cm 단위)**이고 레이블은 **0/1/2 세 가지** 값입니다. 예측 대상이 3개 클래스이므로 이 문제는 **다중 분류 (Multiclass Classification)**에 해당합니다. 레이블이 이미 숫자이므로 별도의 인코딩(2.5장)이 필요 없습니다.

4. 학습 데이터와 테스트 데이터 분리 - train_test_split()

[illegible]

줄	코드	상세 설명
1	X_train, X_test, y_train, y_test = train_test_split(...)	<code>train_test_split()</code> 은 4개의 값을 순서대로 반환합니다: 학습용 피쳐, 테스트용 피쳐, 학습용 레이블, 테스트용 레이블. 이 반환 순서를 혼동하면 학습이 엉뚱하게 되므로 반드시 외워 두어야 합니다.
1	train_test_split(iris_data, iris_label, ...)	첫 번째 인자는 피쳐 데이터 세트 , 두 번째는 레이블 데이터 세트 입니다. 두 데이터는 같은 순서로 대응되어 함께 섞이고 나뉩니다.
2	test_size=0.2	전체 데이터 중 테스트 데이터가 차지할 비율 입니다. 0.2이므로 150건 중 30건이 테스트용 , 나머지 120건이 학습용 이 됩니다. 정수를 주면 비율이 아닌 건수로 해석됩니다.
2	random_state=11	데이터를 나눌 때 사용하는 난수 발생 시드값 입니다. 이 값을 고정하면 몇 번을 실행해도 항상 같은 방식으로 분할 되어 결과를 재현할 수 있습니다. 지정하지 않으면 실행할 때마다 결과가 달라집니다. 숫자 11 자체에는 의미가 없으며 아무 값이나 가능합니다.

▶ 분할 결과 확인

```
X_train.shape: (120, 4)    X_test.shape: (30, 4)
y_train.shape: (120,)      y_test.shape: (30,)
```

△ 왜 데이터를 나누어야 하는가

학습에 사용한 데이터로 성능을 평가하면 **정확도가 100%로 나오지만 아무 의미가 없습니다**. 이미 정답을 본 문제를 다시 푸는 것과 같기 때문입니다. 모델의 진짜 실력은 **한 번도 본 적 없는 데이터**에서 측정해야 합니다. 이 내용은 2.4장에서 자세히 다룹니다.

■ 알아 두면 좋은 옵션 - stratify

`train_test_split(..., stratify=iris_label)` 을 추가하면 **레이블의 클래스 비율을 학습/테스트 세트에 동일하게 유지하며** 분할합니다. 클래스가 불균형한 데이터에서는 거의 필수적인 옵션입니다.

5. 모델 학습 - fit()

```
# DecisionTreeClassifier 객체 생성
dt_clf = DecisionTreeClassifier(random_state=11)

# 학습 수행
dt_clf.fit(X_train, y_train)
```

줄	코드	상세 설명
2	dt_clf = DecisionTreeClassifier(random_state=11)	결정 트리 분류기 객체(Estimator)를 생성합니다. 이 시점에는 아직 학습이 전혀 이뤄지지 않은 빈 껍데기 입니다. <code>random_state</code> 는 트리를 만들 때 동일한 성능의 분할 후보 중 하나를 고르는 과정 등에 쓰이는 시드로, 결과 재현성 을 위해 지정합니다.
5	dt_clf.fit(X_train, y_train)	학습을 수행하는 핵심 메서드 입니다. 첫 인자에 학습용 피쳐 , 두 번째에 학습용 레이블 을 전달합니다. 내부적으로 결정 트리 알고리즘이 120개 데이터를 분석해 "꽃잎 길이가 2.45 이하면 setosa" 같은 규칙 트리를 만들어 객체 안에 저장합니다. 반환값은 학습된 자기 자신(estimator) 이라 주피터에서는 객체 표현식이 출력됩니다.

▶ 실행 결과

```
DecisionTreeClassifier(random_state=11)
```

■ 사이킷런 Estimator의 일관된 인터페이스

사이킷런의 **모든 지도학습 알고리즘**은 `fit( )` 과 `predict( )` 라는 동일한 메서드 이름을 사용합니다. 따라서 `DecisionTreeClassifier` 를 `RandomForestClassifier` 나 `LogisticRegression` 으로 **한 줄만 바꿔도 나머지 코드는 그대로** 동작합니다. 이 설계 덕분에 여러 알고리즘을 손쉽게 비교할 수 있습니다(2.6장에서 실습).

6. 예측 수행 - predict()

```
# 학습이 완료된 DecisionTreeClassifier 객체에서 테스트 데이터 세트로 예측 수행.
pred = dt_clf.predict(X_test)
```

줄	코드	상세 설명
---	----	-------

2	<code>pred = dt_clf.predict(X_test)</code>	학습된 모델에 테스트용 피쳐(X_test) 만 입력하여 레이블을 예측합니다. y_test 는 절대 넣지 않습니다 — 정답을 알려 주면 예측이 아니기 때문입니다. 반환값은 30개 예측 레이블이 담긴 ndarray 이며, 입력 데이터의 순서와 1:1 로 대응합니다.
---	--	--

▶ 실행 결과 (pred 값)

```
[2 2 1 1 2 0 1 0 0 1 1 1 1 2 2 0 2 1 2 2 1 0 0 1 0 0 2 1 0 1]
```

■ predict() vs predict_proba()

`predict()` 는 최종 예측 클래스(0/1/2)를 반환하지만, `predict_proba()` 는 각 클래스에 속할 확률을 반환합니다. 예를 들어 `[0.0, 0.9, 0.1]` 이면 1번 클래스일 확률이 90%라는 의미이며, 임계값을 조정하고 싶을 때 활용합니다.

7. 성능 평가 - accuracy_score()

```
from sklearn.metrics import accuracy_score
print('예측 정확도: {0:.4f}'.format(accuracy_score(y_test, pred)))
```

줄	코드	상세 설명
1	<code>from sklearn.metrics import accuracy_score</code>	<code>sklearn.metrics</code> 는 성능 평가 지표를 제공하는 모듈입니다. <code>accuracy_score</code> 는 정확도를 계산합니다.
2	<code>accuracy_score(y_test, pred)</code>	첫 인자에 실제 정답(y_test), 두 번째에 예측값(pred)을 넣습니다. 순서가 정해져 있으니 주의하세요. 계산식은 맞힌 개수 ÷ 전체 개수 입니다. 30건 중 28건을 맞혀 0.9333이 나왔습니다.
2	<code>'{0:.4f}'.format(...)</code>	문자열 포매팅으로 소수점 넷째 자리까지 표시합니다. <code>{0}</code> 은 첫 번째 인자, <code>:.4f</code> 는 실수 4자리 표기를 의미합니다.

▶ 실행 결과

```
예측 정확도: 0.9333
```

■ 결과 해석

정확도 93.33%는 테스트 데이터 30건 중 **28건을 맞히고 2건을 틀렸다는** 뜻입니다($28/30 = 0.9333$). 붓꽃 데이터는 클래스가 균등하게 분포되어 있어 정확도만으로도 성능을 판단할 수 있지만, 클래스가 불균형한 데이터에서는 정확도가 왜곡될 수 있어 정밀도·재현율·F1 등 다른 지표를 함께 봐야 합니다.

⚠ 이 결과를 그대로 믿어도 될까?

테스트 데이터 30건은 표본이 작아 **어떻게 분할되었는지(random_state)**에 따라 정확도가 크게 흔들립니다. 이 문제를 해결하는 방법이 **교차 검증(Cross Validation)**이며, 2.4장에서 자세히 다룹니다.

8. 핵심 요약 및 머신러닝 활용 포인트

8.1 전체 코드 흐름 정리

```
from sklearn.datasets import load_iris
from sklearn.tree import DecisionTreeClassifier
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score

# ① 데이터 세트 로딩
iris = load_iris()

# ② 학습/테스트 데이터 분리
X_train, X_test, y_train, y_test = train_test_split(
    iris.data, iris.target, test_size=0.2, random_state=11)

# ③ 모델 생성 및 학습
dt_clf = DecisionTreeClassifier(random_state=11)
dt_clf.fit(X_train, y_train)

# ④ 예측
pred = dt_clf.predict(X_test)

# ⑤ 평가
print('예측 정확도: {0:.4f}'.format(accuracy_score(y_test, pred)))
```

8.2 핵심 API 요약표

API	역할	주요 인자 / 반환
<code>load_iris()</code>	붓꽃 데이터 로딩	Bunch 객체 반환 (data, target, feature_names, target_names)
<code>train_test_split()</code>	학습/테스트 분리	반환 순서: X_train, X_test, y_train, y_test test_size, random_state, stratify, shuffle
<code>DecisionTreeClassifier()</code>	결정 트리 분류기 생성	random_state, max_depth, min_samples_split 등
<code>fit(X, y)</code>	모델 학습	학습용 피처와 레이블을 함께 전달
<code>predict(X)</code>	예측 수행	피처만 전달, 예측 레이블 ndarray 반환
<code>accuracy_score(y, pred)</code>	정확도 계산	실제값이 첫 인자 , 예측값이 두 번째

8.3 다음 장으로 이어지는 학습 포인트

- **2.3장** : 방금 사용한 `load_iris( )` 의 반환 객체(Bunch)와 사이킷런 Estimator의 구조를 자세히 살펴봅니다.
- **2.4장** : 단 한 번의 분할로 얻은 정확도의 한계를 **교차 검증**으로 보완하고, **GridSearchCV**로 최적 하이퍼 파라미터를 찾습니다.
- **2.5장** : 붓꽃 데이터는 이미 숫자로 정리되어 있었지만, 실제 데이터는 **인코딩과 스케일링**이 필요합니다.
- **2.6장** : 결측치와 문자열이 섞인 **타이타닉 실전 데이터**에 이 모든 과정을 적용해 봅니다.

⚠ 안정화 버전 참고 (scikit-learn 1.7.x 기준)

- 본 장의 API는 사이킷런 0.2x 버전부터 현재까지 **변경 없이 동일**하게 동작합니다.
- `load_iris(as_frame=True)` 옵션을 쓰면 ndarray 대신 **DataFrame**으로 바로 로딩할 수 있어 편리합니다.
- 주피터 환경에서 `fit( )` 호출 결과는 버전에 따라 **텍스트** 또는 **대화형 HTML 다이어그램**으로 표시될 수 있습니다. 표시 형태만 다를 뿐 동작은 동일합니다.